Decision Tree Classification using Iris Dataset

📌 Project Overview

This project demonstrates how **Decision Tree Machine Learning Algorithm** can be used to classify flower species from the famous **Iris dataset**.

Decision Trees are one of the most **interpretable and powerful supervised learning algorithms** used for both **classification and regression tasks**.

In this notebook we will:

- Load and explore the Iris dataset
- Perform **data visualization**
- Split the dataset into **training and testing sets**
- Train a **Decision Tree Classifier**
- Evaluate model performance
- Visualize the **decision tree structure**

🎯 Objective

The objective of this project is to build a **Decision Tree model that predicts the species of Iris flower** based on its features.

📊 Dataset Information

The Iris dataset contains **150 samples** of iris flowers with the following features:

Feature	Description
Sepal Length	Length of the sepal
Sepal Width	Width of the sepal
Petal Length	Length of the petal
Petal Width	Width of the petal
Species	Target variable

Species Classes:

- Setosa

- Versicolor
- Virginica

📚 Importing Required Libraries

Before building a machine learning model, we need to import the **necessary Python libraries**.

Libraries Used

Pandas

- Used for **data manipulation and analysis**
- Helps in working with structured datasets

NumPy

- Provides support for **numerical computations**
- Efficient for working with arrays and mathematical operations

These libraries form the **foundation of most data science workflows**.

In [1]:
```
# importing major libraries
import pandas as pd
import numpy as np
```

🌸 Loading the Iris Dataset

In this step we load the **Iris dataset** using the `seaborn` library.

Why Iris Dataset?

The Iris dataset is one of the **most famous datasets in machine learning** because:

- It is **simple and clean**
- It contains **three flower species**
- It is commonly used for **classification problems**

Dataset Features

The dataset includes measurements of:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

The goal is to **predict the species of the flower** using these features.

In [2]:
```
# Iris
import seaborn as sns
df=sns.load_dataset('iris')
df.head()
```

Out[2]:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

📊 Data Visualization

Data visualization helps us **understand patterns and relationships** between variables.

Here we use **Seaborn Pairplot** to visualize how different features relate to each other.

Why Pairplot?

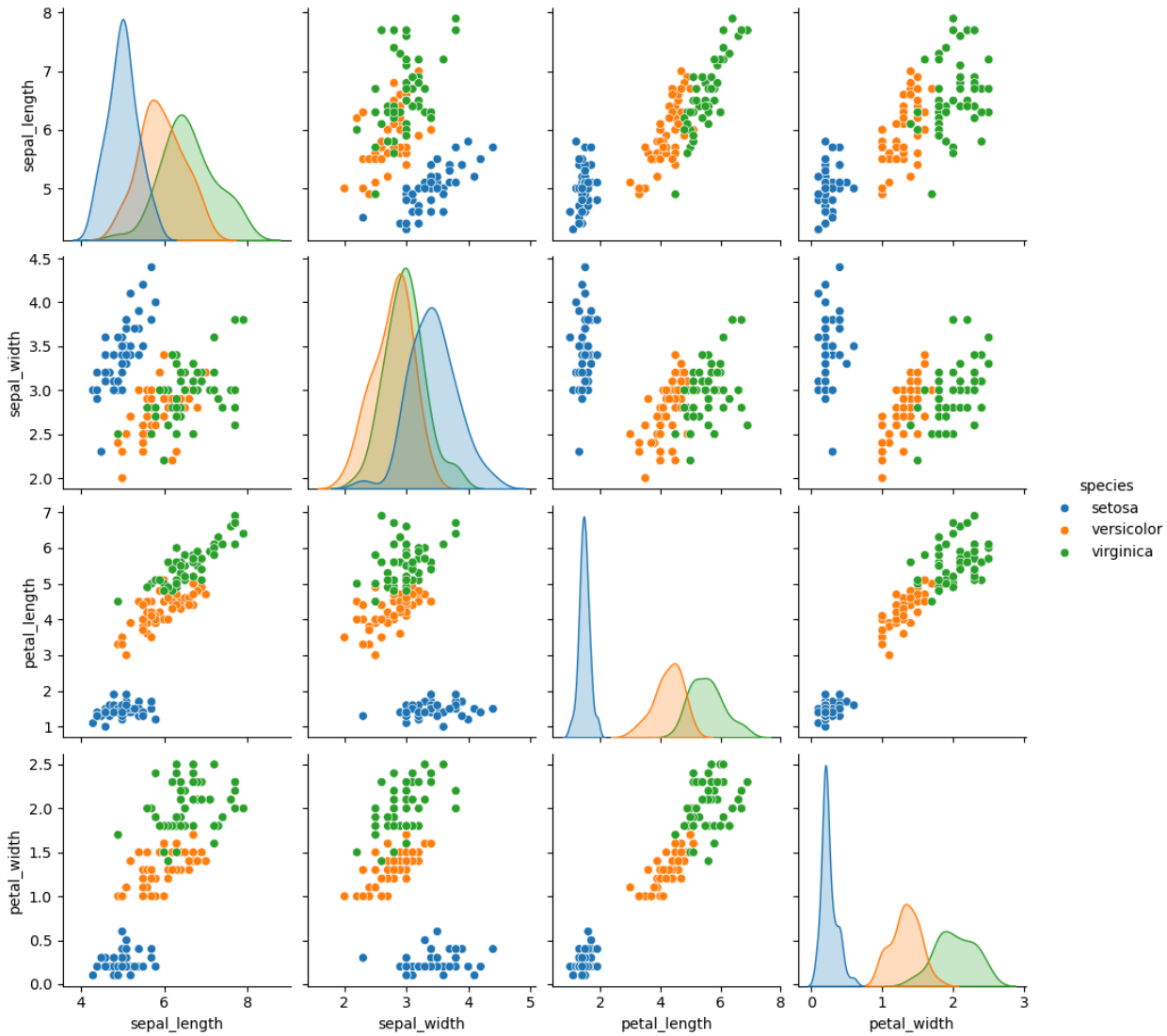
Pairplot allows us to:

- Compare all numerical features together
- Observe **class separation**
- Identify patterns between species

The `hue` parameter colors the data points according to the **flower species**.

In [3]:
```
sns.pairplot(df,hue='species')
```

Out[3]:
```
<seaborn.axisgrid.PairGrid at 0x2200e3f3a10>
```



🎯 Feature Selection

Machine learning models require two main components:

Independent Variables (X)

These are the **input features** used to make predictions.

In this dataset:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

Dependent Variable (Y)

This is the **target variable** the model tries to predict.

In this case:

- **Species of the Iris flower**

In [4]:
```
x=df.iloc[:,0:-1]
y=df.species
```

🔀 Splitting the Dataset

To evaluate the performance of a machine learning model properly, we split the dataset into two parts:

Training Data

Used to **train the machine learning model**

Testing Data

Used to **evaluate the model's performance on unseen data**

We use:

`train_test_split()` from **scikit-learn**

Parameters Used

- `test_size = 0.2` → 20% data used for testing
- `random_state = 42` → ensures reproducibility

In [5]:
```
#train test split
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.2,random_stat
```

Decision Tree Algorithm

A **Decision Tree** is a supervised machine learning algorithm used for:

- Classification
- Regression

It works by **splitting data into branches based on feature values**.

Key Idea

The model repeatedly asks questions like:

- Is petal length greater than a certain value?
- Is sepal width smaller than a threshold?

Based on these decisions, the tree splits the data until it reaches a **final prediction (leaf node)**.

Criterion Used

We use **Gini Impurity** to measure the quality of splits.

Lower Gini value → Better split

In [6]:
```
from sklearn.tree import DecisionTreeClassifier
dtc=DecisionTreeClassifier(criterion='gini')
```

In [7]:
```
dtc
```

Out[7]:
```
▾ DecisionTreeClassifier
DecisionTreeClassifier()
```

⚙ Training the Model

In this step we train the Decision Tree model using the **training dataset**.

Training Process

The model learns patterns from the data by:

1. Finding the best feature to split
2. Creating branches based on conditions
3. Repeating the process until optimal classification is achieved

The `.fit()` function trains the model using:

- Training features
- Training labels

In [8]:
```
dtc.fit(x_train,y_train)
```

Out[8]:
```
▾ DecisionTreeClassifier
DecisionTreeClassifier()
```

📈 Model Evaluation

After training the model, we evaluate how well it performs on **unseen data**.

Accuracy Score

Accuracy measures the **percentage of correct predictions** made by the model.

Formula:

Accuracy = Correct Predictions / Total Predictions

Higher accuracy indicates **better model performance**.

In [9]:
```
from sklearn.metrics import accuracy_score
y_pred = dtc.predict(x_test)
accuracy_score(y_test,y_pred)
```

Out[9]:
```
1.0
```

Decision Tree Visualization

One of the biggest advantages of Decision Trees is that they are **highly interpretable**.

We can visualize the entire tree structure to understand:

- How decisions are made
- Which features are most important
- The sequence of splits

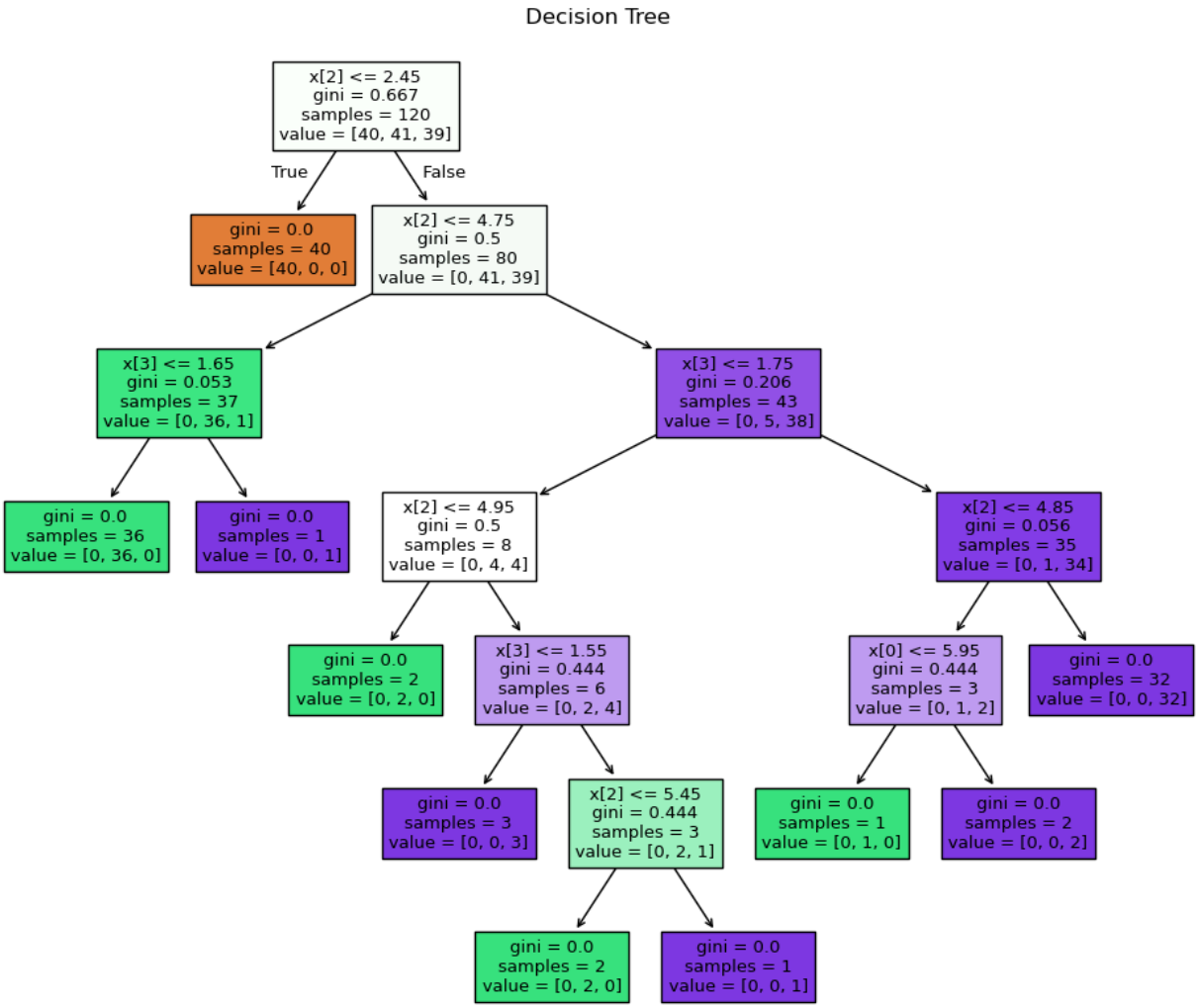
The visualization shows:

- Decision nodes
- Branch conditions
- Final class predictions

This helps in **explaining the model easily to non-technical audiences**.

In [10]:
```
from sklearn import tree
import matplotlib.pyplot as plt

plt.figure(figsize=(12,10))
tree.plot_tree(dtc,filled=True)
plt.title('Decision Tree')
plt.show()
```



🚀 Key Takeaways

✔ Built a Decision Tree Classification Model
✔ Performed Data Visualization using Seaborn
✔ Implemented Train-Test Split
✔ Evaluated model performance using Accuracy Score
✔ Visualized the Decision Tree structure

⛶ Tools & Technologies

- Python
- Pandas
- NumPy
- Seaborn
- Scikit-learn
- Matplotlib

📌 Author

Vipul Pandey

Mathematics Student | Data Analytics Enthusiast | Machine Learning Learner